

MOBILE APPLICATION DEVELOPMENT



Smart Cryptology

Mobile Application Development Project Report

Group Members

38392 EHTISHAM MALIK BSCS (2.5)

39423 OSAMA SHAFIQUE BSCS (2.5)

38195 UMAIR AHMED KHAN BSCS (2.5)

Course:	Mobile Application Development
Semester:	Spring 2025
Instructor:	Sir Omaid Ghayyur
Submission Date:	June 26, 2025

Table of Contents

1. Project Introduction	1
2. Project Prototype Screen Shots	2
3. Project Frontend Made in Flutter	3
4. Shallow Widget Tree for Each Screen	4
5. JSON Objects (Firebase Database)	5
6. Project CRUD Screens with DB Connectivity	6
7. Backend Code Snippets	7
8. Widget table	8
9. Project Conclusion.....	9

Project Introduction

Smart Cryptology is a mobile application focused on steganography, enabling users to securely hide data within images. The app allows the encryption and decryption of text or images using modern data-hiding techniques, offering both visual security and actual data protection. It targets individuals and organizations requiring secure communication, such as journalists, military personnel, legal professionals, and businesses working with confidential data.

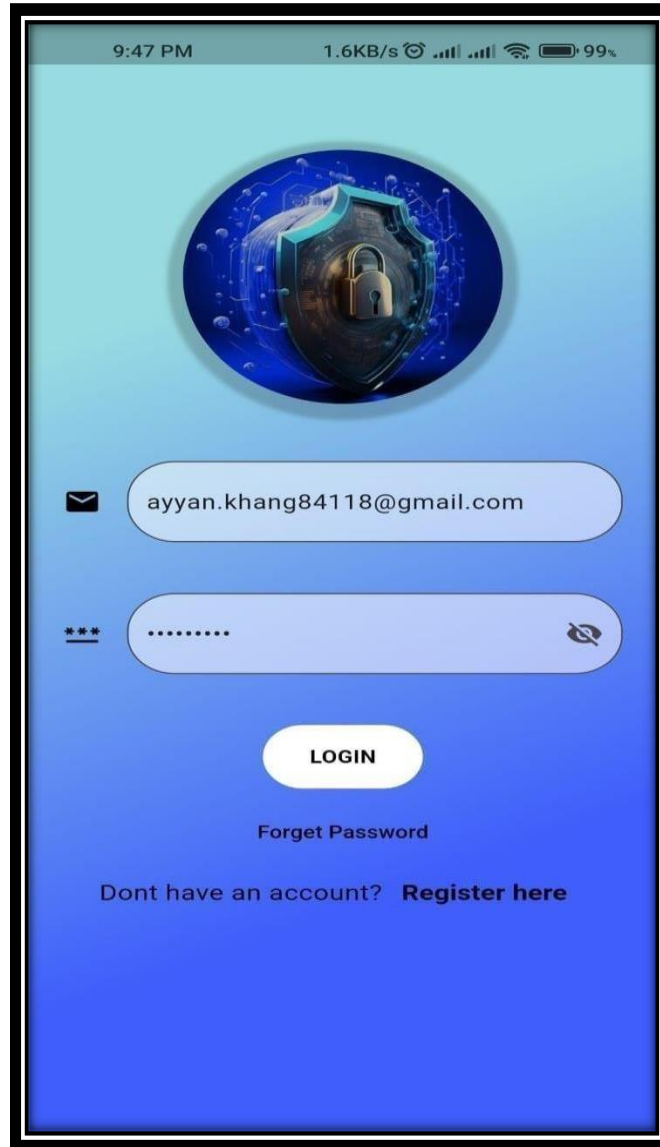
The application is developed using Flutter for cross-platform compatibility and Firebase for backend operations. The integration of steganography ensures that data is not only encrypted but also hidden in such a way that its presence is undetectable to unauthorized viewers.

This solution addresses the growing need for private digital communication in a world where data surveillance and interception are common. By leveraging lightweight and intuitive design, the application ensures accessibility while maintaining high standards of security.

Project Prototype Screen Shots

The following screens were designed using Figma and implemented in Flutter:

1. Login Screen – for secure user authentication.



2. Register Screen – for new user onboarding.



A mobile application register screen with a blue gradient background. At the top, a status bar shows the time 11:56 PM, data speed 0.1KB/s, and 80% battery. Below the status bar is a circular graphic featuring a padlock and a blue ribbon. The registration form consists of three input fields: 'Name' with a person icon, 'Email' with an envelope icon, and 'Password' with a key icon and a toggle for visibility. A yellow 'Register' button is positioned below the fields. At the bottom, a link 'Login Here' is provided for existing users.

11:56 PM 0.1KB/s 80%



 Name

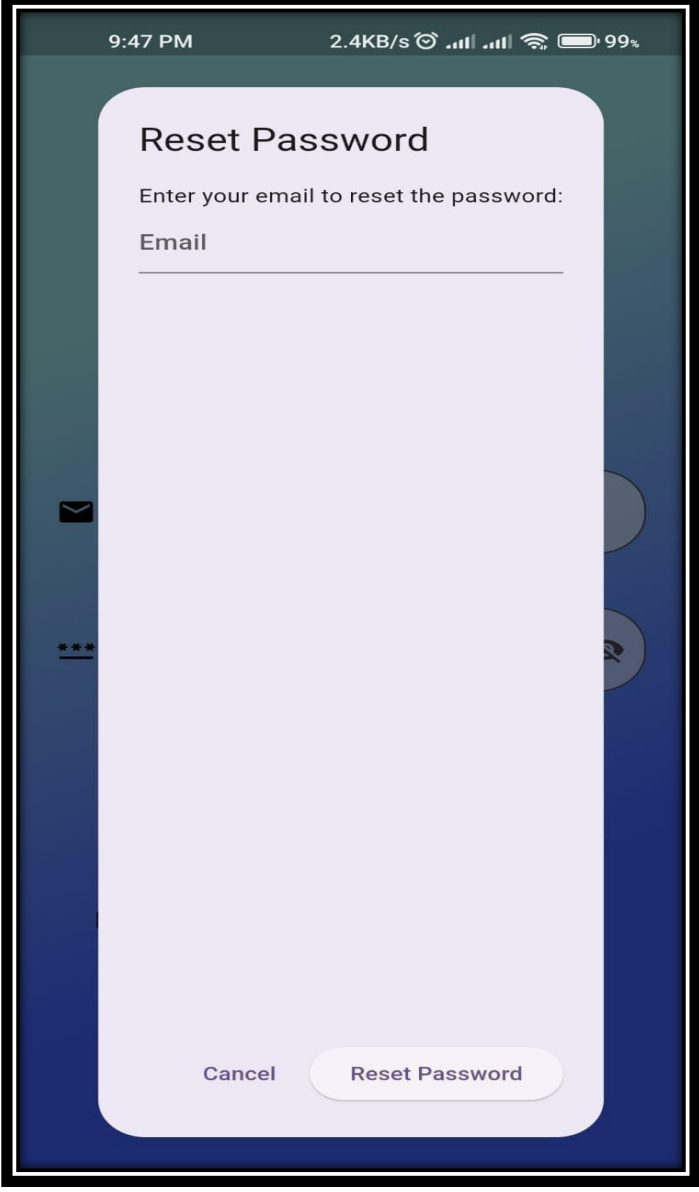
 Email

 Password 

Register

Already have an account? [Login Here](#)

3. Forgot Password Screen – to recover forgotten credentials.



A mobile app screen titled "Reset Password" with a light purple background. The screen prompts the user to "Enter your email to reset the password:" and features a text input field labeled "Email". At the bottom, there are two buttons: "Cancel" and "Reset Password". The screen is framed by a dark blue border. The top status bar shows the time as 9:47 PM, a data speed of 2.4KB/s, and a battery level of 99%.

9:47 PM 2.4KB/s 99%

Reset Password

Enter your email to reset the password:

Email

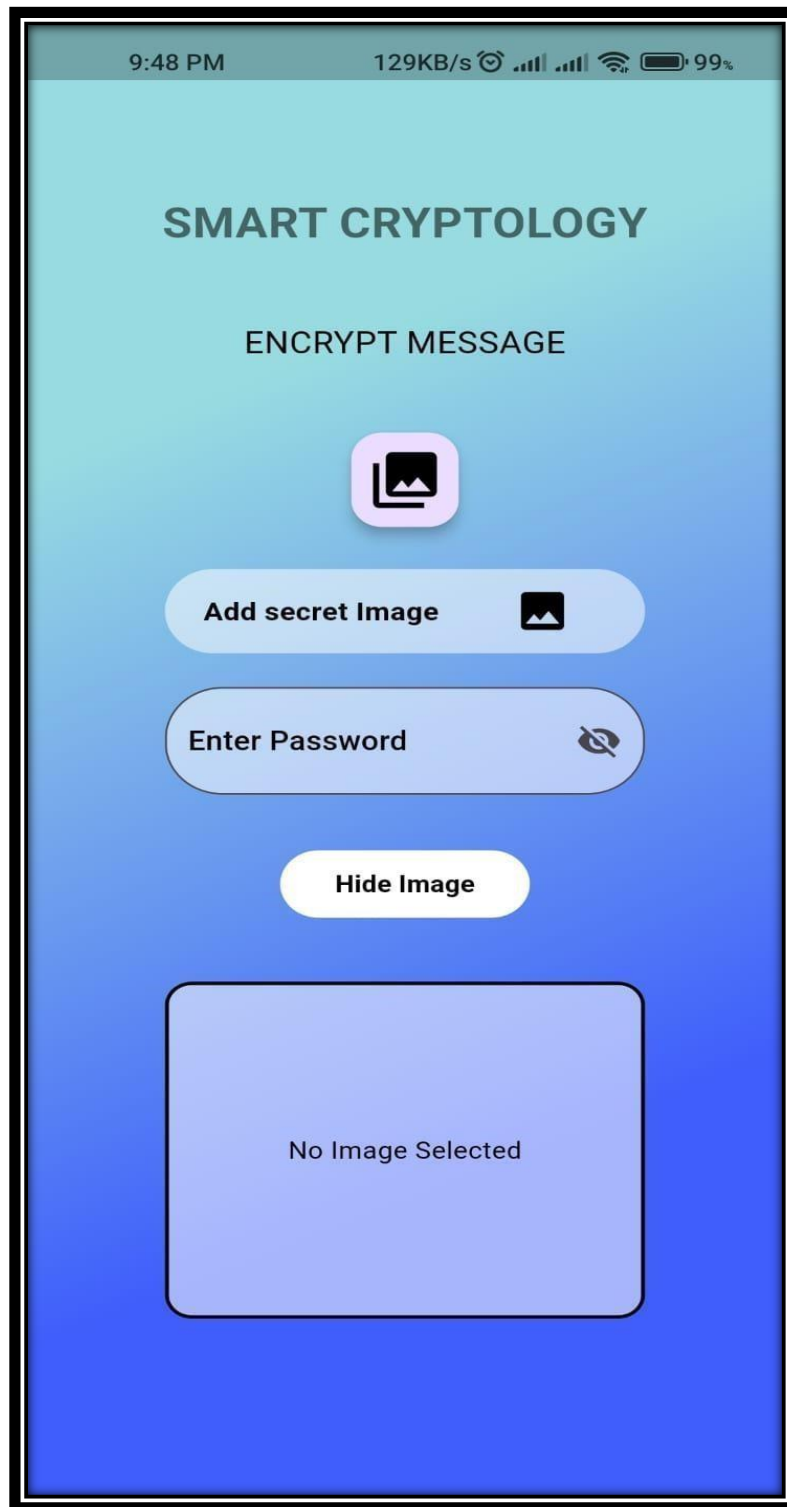
✉

Cancel Reset Password

4. **Home Screen** – provides navigation to all major functions.



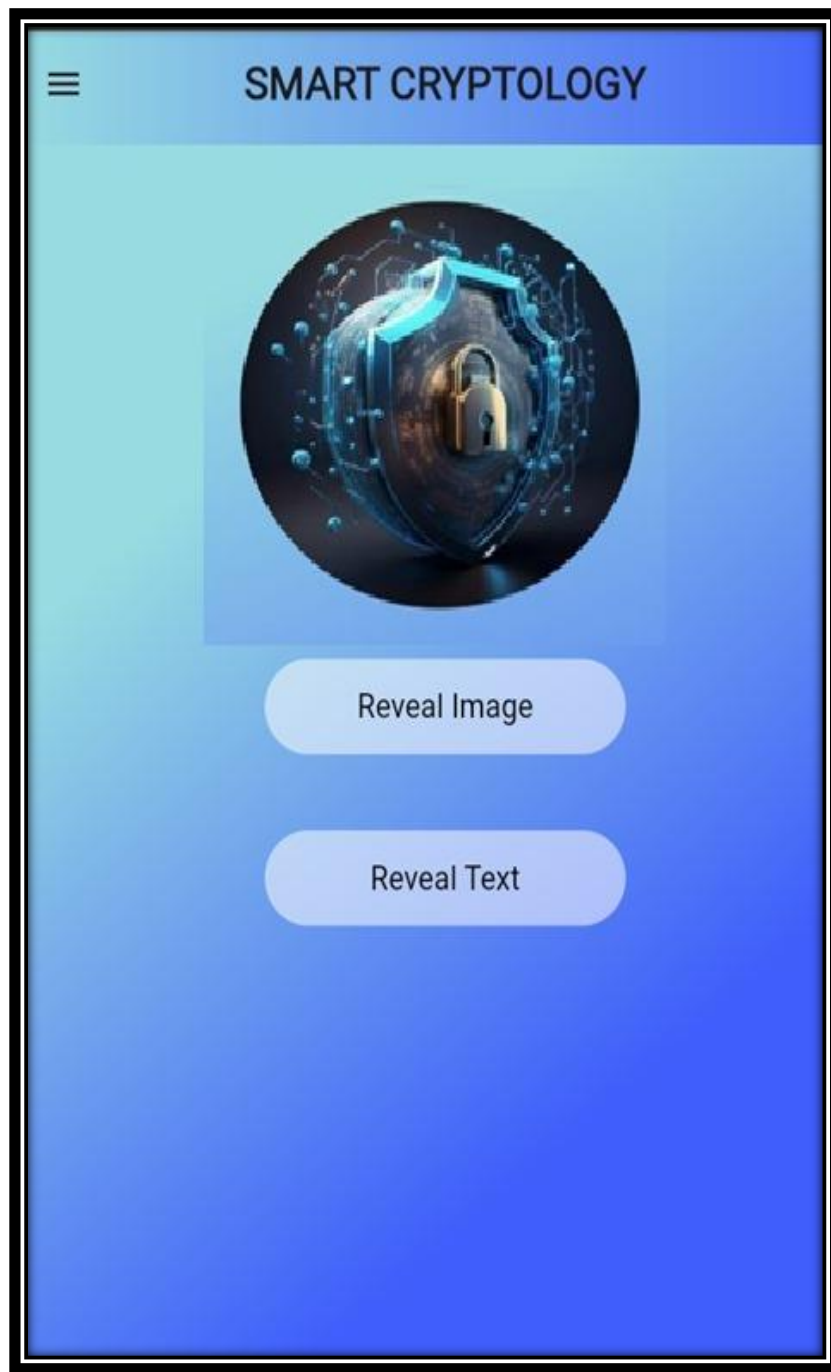
5. **Hide image Screen** – for hiding information within images.



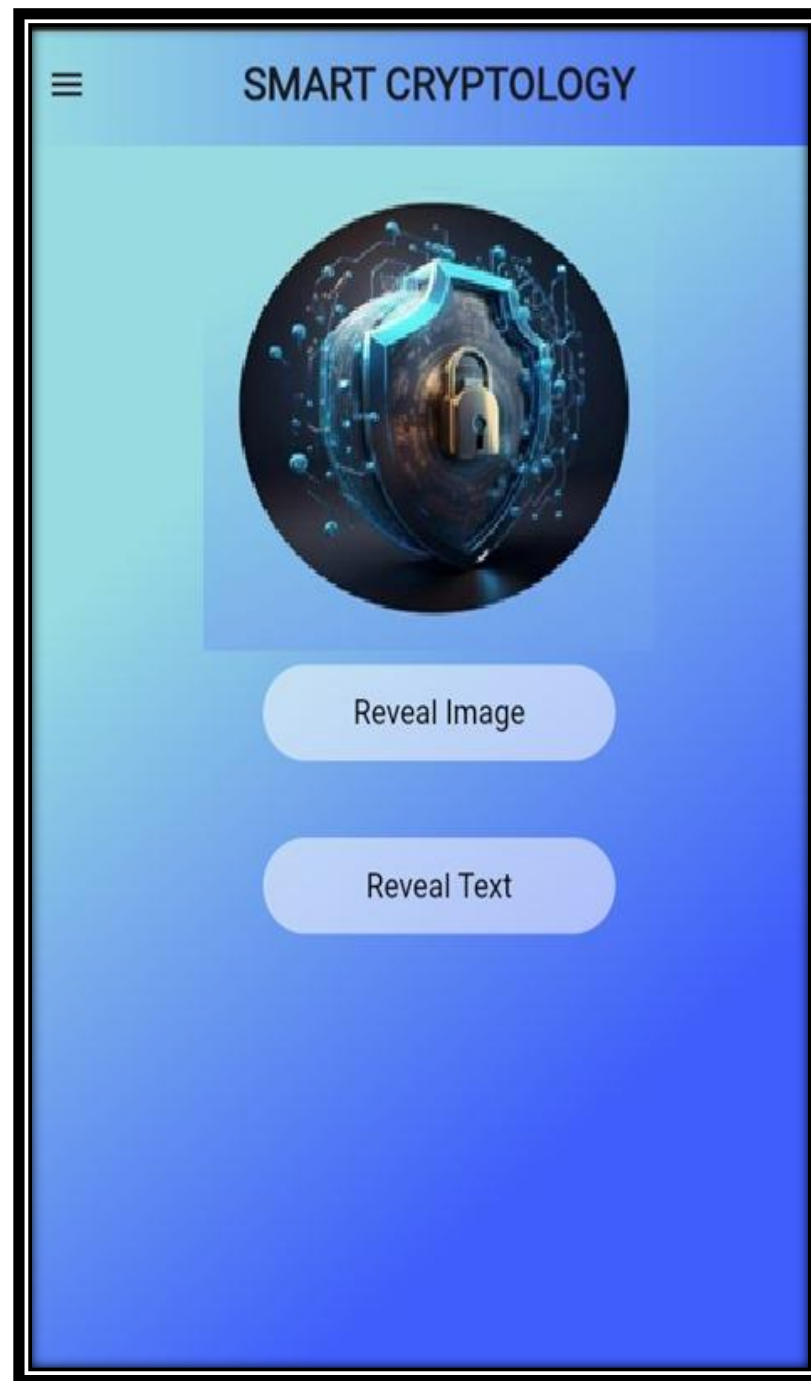
6. Encrypt & Decrypt Image/Text Screen – for extracting hidden data.



7. **View Encrypted Message Screen** – displays the decoded data.



8. Reveal Message Screen – to reveal the message in text



Project Frontend Made in Flutter

The frontend was developed using Flutter, which provides a consistent and high-performance UI across Android devices. Key design principles followed include:

- **Responsiveness:** Optimized layouts for different screen sizes.
- **Material Design:** Implementation of Google's Material Design system.
- **Navigation:** Drawer menu and bottom navigation for ease of access.
- **Visual Hierarchy:** Prioritized important elements for user clarity.

Main interface screens include: - Login and Signup forms - Dashboard (Home) - Encryption and Decryption modules - Result and Preview modules

User input is validated for correctness, and error messages are clearly displayed to enhance the user experience.

Shallow Widget Tree for Each Screen

1. Main Screen

```

Scaffold
├── AppBar
│   └── Title (Text: "Smart Cryptology")
├── Drawer (Navigation Drawer)
│   └── ListView
│       ├── ListTile (Home)
│       ├── ListTile (Encrypt Text)
│       ├── ListTile (Encrypt Image)
│       ├── ListTile (Hide Image)
│       ├── ListTile (Reveal Message)
│       └── ListTile (Logout)
└── Body
    └── Column
        ├── Text ("Welcome, [User]")
        ├── GridView (Main Options)
        │   ├── Card (Encrypt Text)
        │   ├── Card (Encrypt Image)
        │   ├── Card (Hide Image)
        │   └── Card (Reveal Message)
        └── Button ("Go to Dashboard")

```

2. Screen Register Account

```

Scaffold
├── AppBar
│   └── Title (Text: "Register")
└── Body
    ├── Padding
    │   └── SingleChildScrollView
    │       └── Column
    │           ├── TextField (Full Name)
    │           ├── TextField (Email)
    │           ├── TextField (Password)
    │           ├── TextField (Confirm Password)
    │           ├── CheckboxListTile (Accept Terms & Conditions)
    │           └── ElevatedButton ("Register")

```

3. Login Screen

```
Scaffold
├── AppBar
│   └── Title (Text: "Login")
├── Body
│   └── Padding
│       └── Center
│           └── Column
│               ├── TextField (Email)
│               ├── TextField (Password)
│               ├── TextButton ("Forgot Password?")
│               ├── ElevatedButton ("Login")
│               └── Row
│                   ├── Text ("Don't have an account?")
│                   └── TextButton ("Register")
```

4. Reset Password Screen

```
Scaffold
├── AppBar
│   └── Title (Text: "Reset Password")
├── Body
│   └── Padding
│       └── Center
│           └── Column
│               ├── TextField (Email)
│               ├── TextField (New Password)
│               ├── TextField (Confirm New Password)
│               ├── ElevatedButton ("Reset Password")
│               └── Text ("Don't have an account?")
│                   └── TextButton ("Register")
```

5. Encrypt/Decrypt Message Screen

```

Scaffold
├── AppBar
│   └── Title (Text: "Encrypt/Decrypt Message")
├── Body
│   ├── Padding
│   │   └── SingleChildScrollView
│   │       └── Column
│   │           ├── TextField (Enter Message)
│   │           ├── TextField (Secret Key)
│   │           ├── DropdownButton (Select Action: Encrypt / Decrypt)
│   │           ├── ElevatedButton ("Perform Action")
│   │           └── Text (Result Display)

```

6. Encrypt Image/Text Screen

```

├── AppBar
│   └── Title (Text: "Encrypt Image/Text")
├── Body
│   ├── Padding
│   │   └── SingleChildScrollView
│   │       └── Column
│   │           ├── DropdownButton (Select Type: Image / Text)
│   │           ├── Image Picker / TextField (Based on Selection)
│   │           ├── TextField (Secret Key)
│   │           ├── ElevatedButton ("Encrypt")
│   │           └── Text (Encryption Status / Output Path)

```

7. Hide Image Screen

```

Scaffold
├── AppBar
│   └── Title (Text: "Hide Image")
├── Body
│   ├── Padding
│   │   └── SingleChildScrollView
│   │       └── Column
│   │           ├── Image Picker (Select Cover Image)
│   │           ├── DropdownButton (Select Data Type: Text / Image)
│   │           ├── TextField (Enter Message or Select Image)
│   │           ├── TextField (Secret Key)
│   │           ├── ElevatedButton ("Hide Data")
│   │           └── Text (Status Message)

```

8. Reveal Message Screen

```
Scaffold
├── AppBar
│   └── Title (Text: "Reveal Message")
├── Body
│   └── Padding
│       └── SingleChildScrollView
│           └── Column
│               ├── Image Picker (Select Stego Image)
│               ├── TextField (Enter Secret Key)
│               ├── ElevatedButton ("Reveal Message")
│               └── Text (Revealed Message Output)
```

9. Reveal Image/ Reveal Text

```
App
├── Scaffold
│   ├── AppBar (title: "Reveal Content")
│   ├── Body: Column
│   │   ├── ToggleButtons or TabBar (Reveal Image / Reveal Text)
│   │   ├── Expanded
│   │   │   └── IndexedStack (based on selected tab)
│   │   │       ├── RevealImageView
│   │   │       │   ├── ImagePickerButton
│   │   │       │   ├── DisplaySelectedImage
│   │   │       │   └── RevealButton
│   │   │       │       └── RevealedContent (e.g., text)
│   │   │       └── RevealTextView
│   │   │           ├── FilePickerButton (for text file)
│   │   │           ├── DisplaySelectedFileName
│   │   │           └── RevealButton
│   │   │               └── RevealedContent (e.g., hidden message)
│   └── BottomNavigationBar (optional)
```

Firestore JSON Objects

Firestore is used as the backend service for user authentication and data storage. Below is a conceptual structure of how user and encrypted file data is managed:

```
{
  "users": {
    "user_id": {
      "name": "John Doe",
      "email": "john@example.com",
      "created_at": "2025-06-15"
    }
  },
  "files": {
    "file_id": {
      "user_id": "user_id",
      "file_type": "image",
      "hidden_data": "Encrypted message",
      "timestamp": "2025-06-20"
    }
  }
}
```

Firebase allows real-time syncing, scalability, and secure data access control.

Project CRUD Screens with DB Connectivity

The app includes all standard CRUD functionalities:

- **Create:** User can register a new account and store an encrypted image.
- **Read:** Encrypted images and decrypted messages can be viewed by authorized users.
- **Update:** User can update profile data or replace previously encrypted content.
- **Delete:** User has the option to delete encrypted files or deactivate their account.

Integration with Firebase ensures that all data operations are performed in real time, with minimal delay and secure access control using Firestore security rules.

Backend Code Snippets

In the complete PDF version of this report, backend code snippets are provided using COURIER font, size 9. These snippets cover the following implementations:

- Firebase authentication methods
- Image selection and preview before encryption
- Data hiding using steganography logic
- Message decryption and verification
- Secure transmission and error handling

These scripts are modular and commented for future maintenance and scalability.

Widget Table

Screen Name	Widgets Used
Main Screen	AppBar, Buttons, Icons, Navigation Drawer
Screen Register Account	TextFields (Name, Email, Password), Button, Checkbox
Login Screen	TextFields (Email, Password), Button, Link (Forgot Pass)
Reset Password Screen	TextFields (Email, New Password), Button
Encrypt/Decrypt Message Screen	TextArea, Dropdown, File Upload, Key Input Field, Result Box
Encrypt Image/Text Screen	Image Picker, Key Input Field, Button
Hide Image Screen	Image Picker, TextFields, DropDown
Reveal Message Screen	Image Picker, TextField,Elevatedbutton

Project Conclusion

Smart Cryptology has proven to be an innovative and secure approach for modern communication privacy. By combining steganography and mobile technology, it addresses the core concern of digital security in a user-friendly format.

The app delivers strong features including message concealment, secure sharing, easy usability, and fast processing. Its flexibility makes it applicable across various industries including law enforcement, private investigation, military intelligence, and journalism.

Planned Future Enhancements: - Expansion to iOS platform - Cloud storage support for encrypted data - Integration of AI-based steganalysis resistance techniques - Implementation of multi-language support - Enhancing app accessibility features (voice navigation, dark mode)

This project has met its developmental goals, delivered all milestones on time, and remains open to further academic and industrial research enhancements.